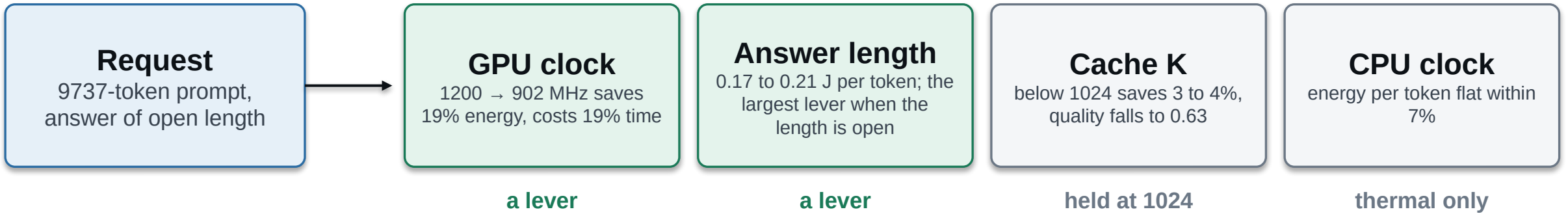
One lever, two loops.
How a phone spends its battery on an LLM request, and what a learned policy adds.

OnePlus 15 · Llama-3.2-1B · every number metered on the device

Md Romyull Islam · Kennesaw State University · September 2026

THE QUESTION

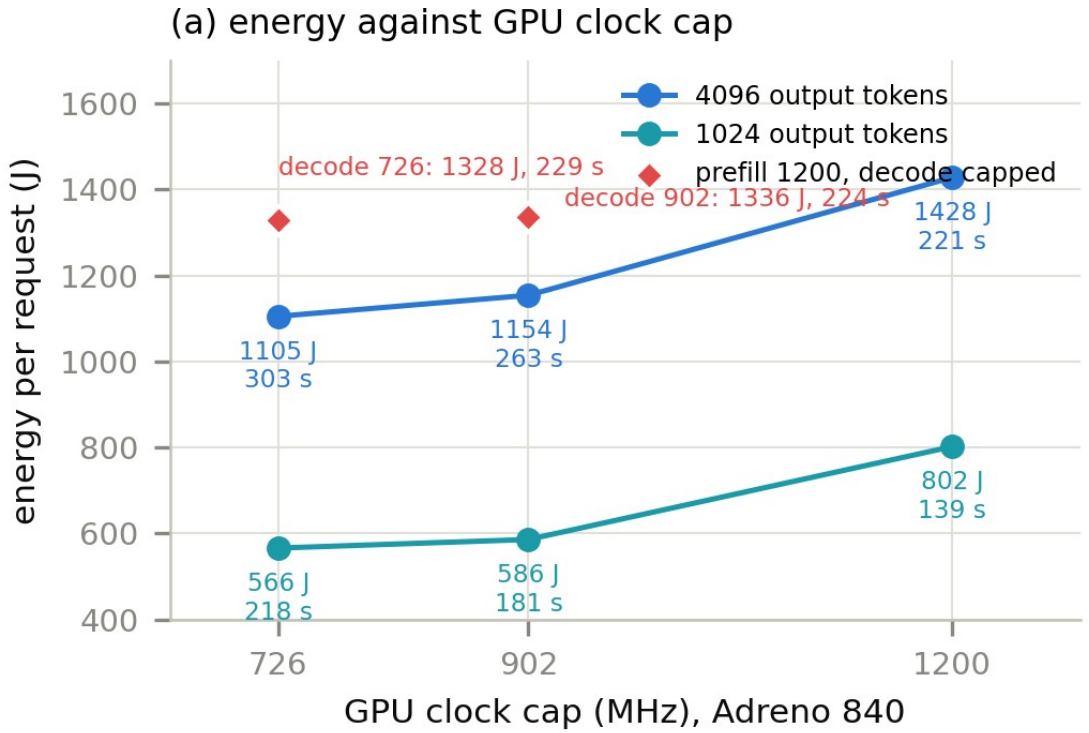
A request arrives. The phone has four knobs

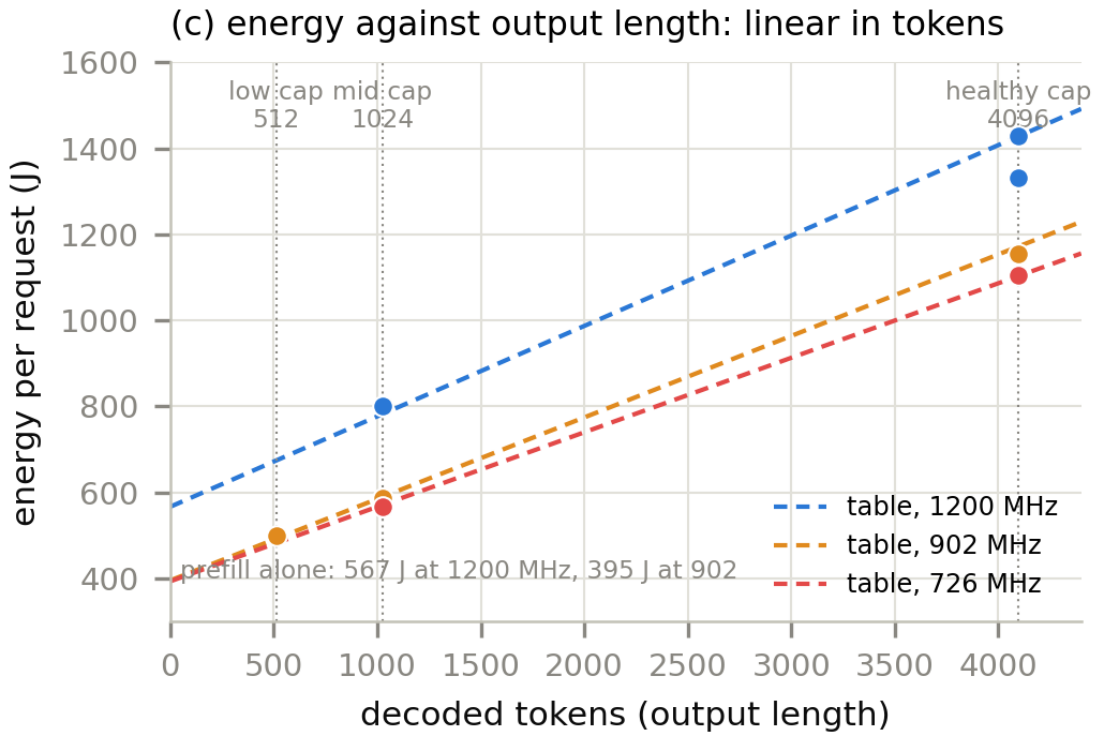


Two knobs carry the saving. Two do not. The phone's own governor reads none of them for an LLM request. No shipped phone changes an LLM's clock, context or answer length by battery state.

Cooled cells, charging off, OnePlus 15 GPU. Details on the next two slides.

GPU clock trades energy for time; answer length is linear





1200 to 902 MHz: 1428 to 1154 J for 221 to 263 s. The last step to 726 buys 3% for 21% more time, so it is never taken.

Every output token costs 0.17 to 0.21 J on top of a fixed prefill. Capping the answer at 1024 or 512 tokens is the biggest saving available.

fig_energy_curves.py, panels (a) and (c). n=3 at 4096 tokens, n=5 to 11 at 1024.

The GPU whenever it works; the CPU is the fallback

plan, 9737-token prompt + 1024 tokens	energy	time
GPU 1200 MHz	782 J	142 s
GPU 902 MHz	589 J	181 s
GPU 726 MHz	571 J	219 s
CPU, K = 1024	**822 J**	**304 s**

CPU plans lose on both axes, so the ladder walk never reaches them while the GPU works.

Bonsai-8B on the CPU	cap 4096	cap 1024	cap 512
measured per-token costs	7853 J	5310 J, -32%	4886 J, -38%

1 No Vulkan library, or the caller forced CPU

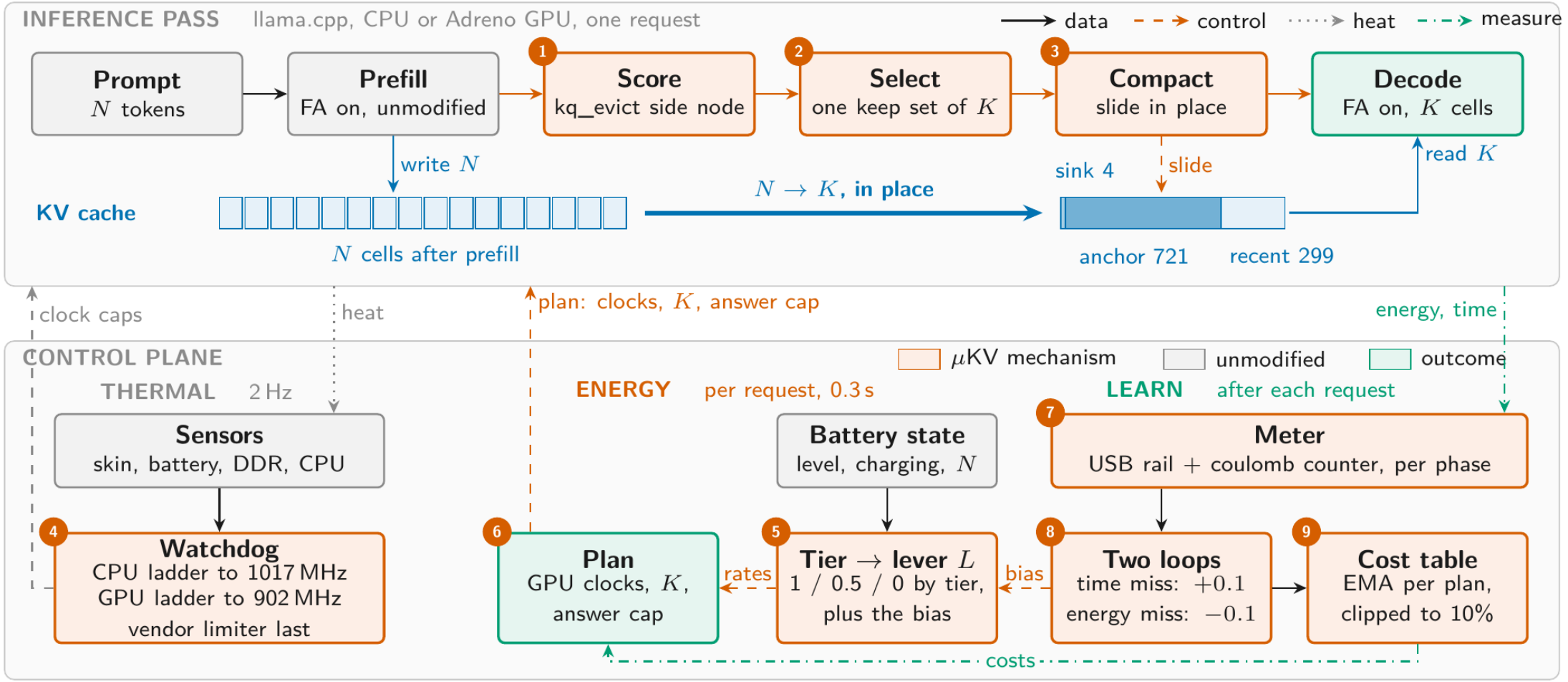
2 The GPU runs it but the output is garbage
PrismML's 1-bit Bonsai-8B: non-finite logits on Adreno, decoded at full speed

Every run is graded. A model that fails is marked, and the request re-runs on the CPU.

On the CPU one lever is left
the clock saves nothing, K cannot drop below 1024, so the answer cap tiers alone

GPU and CPU plan costs predicted from the measured cost table. Bonsai figures from its own per-token costs, /tmp/nat_bonsai. Backend rules in ukv_sched.sh v2.5.

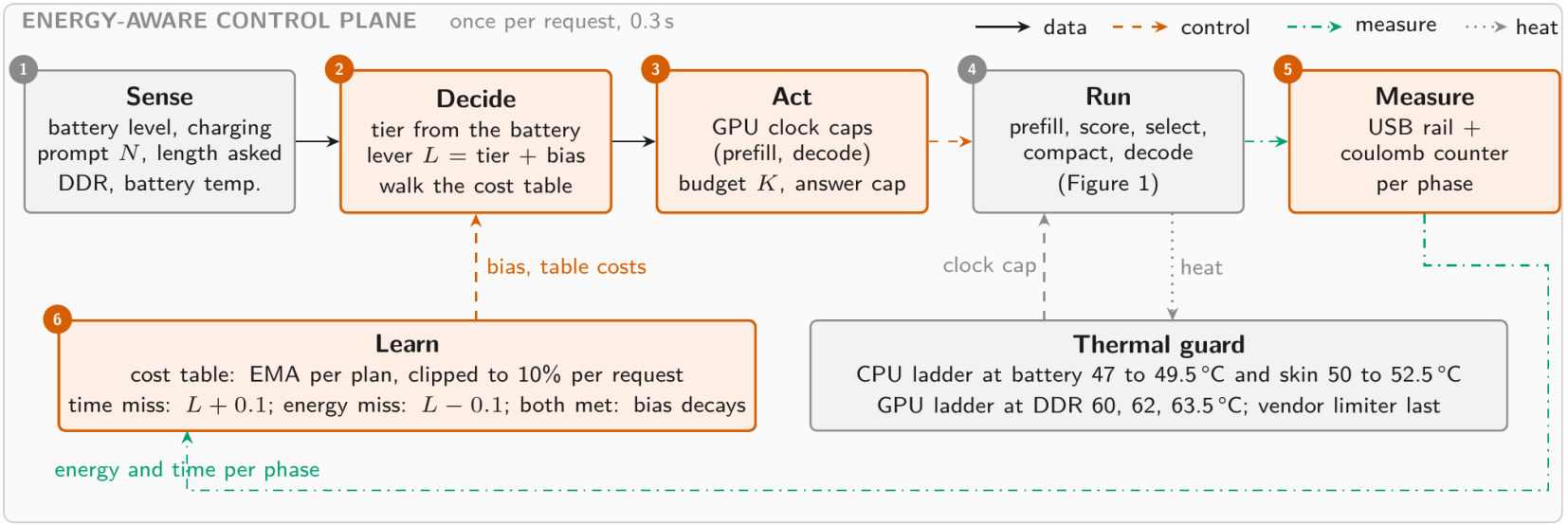
Where the scheduler sits: below an unchanged inference pass



Top: the pass; steps 1 to 3 are the only changes. Bottom: the thermal, energy and learning columns, 4 to 9. Dashed control, dotted heat, dash-dot measurement.

fig_architecture_tikz, the same figure as Figure 1 of the paper.

Once per request, in 0.3 s



Sense
battery, charging, prompt, length asked

Decide
tier → lever → walk the cost table

Act
GPU clocks, cache K, answer cap

Measure, learn
meter → table and lever

ukv_sched.sh v2.3 on the phone. The thermal guard acts on the clock beside it.

THE LEVER

The battery tier sets one number; it sets everything else

mains or above 50%	21 to 50%	20% or below
L = 1	L = 0.5	L = 0

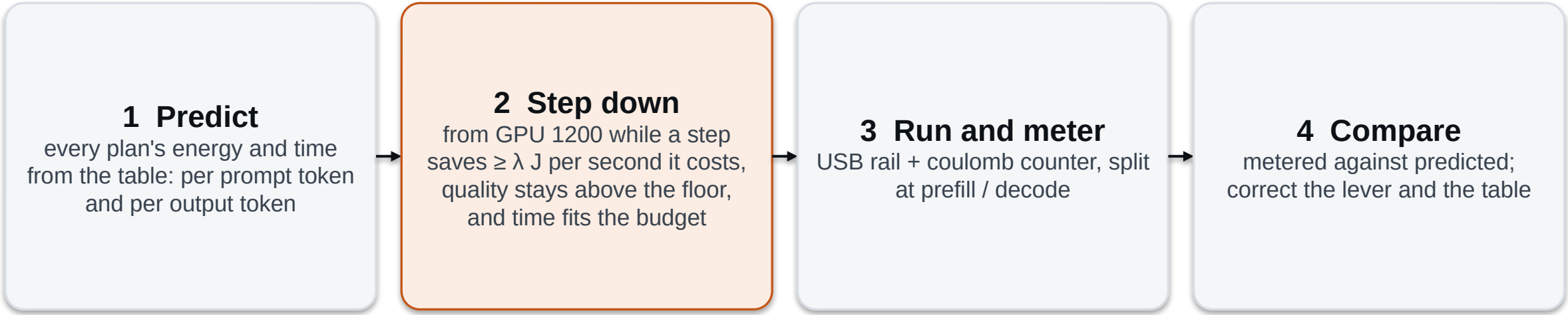
what L sets	L = 1	L = 0.5	L = 0
exchange rate: joules a step must save per second it costs	1.5	0.8	0.5
quality floor	1.00	0.90	0.75
time slack over the fastest plan	5%	21.5%	40%
answer cap when the length is open	4096	1024	512

Full battery: a step down must pay well and may cost little time. Empty battery: almost any saving is taken.

ukv_sched.sh: LAM=pl(0.5,0.8,1.5), QF=pl(0.75,0.90,1.00), TSLACK=0.03+0.37(1−L) floored at 0.05, CAPL by L.

THE WALK

Predict, step down while it pays, run, compare

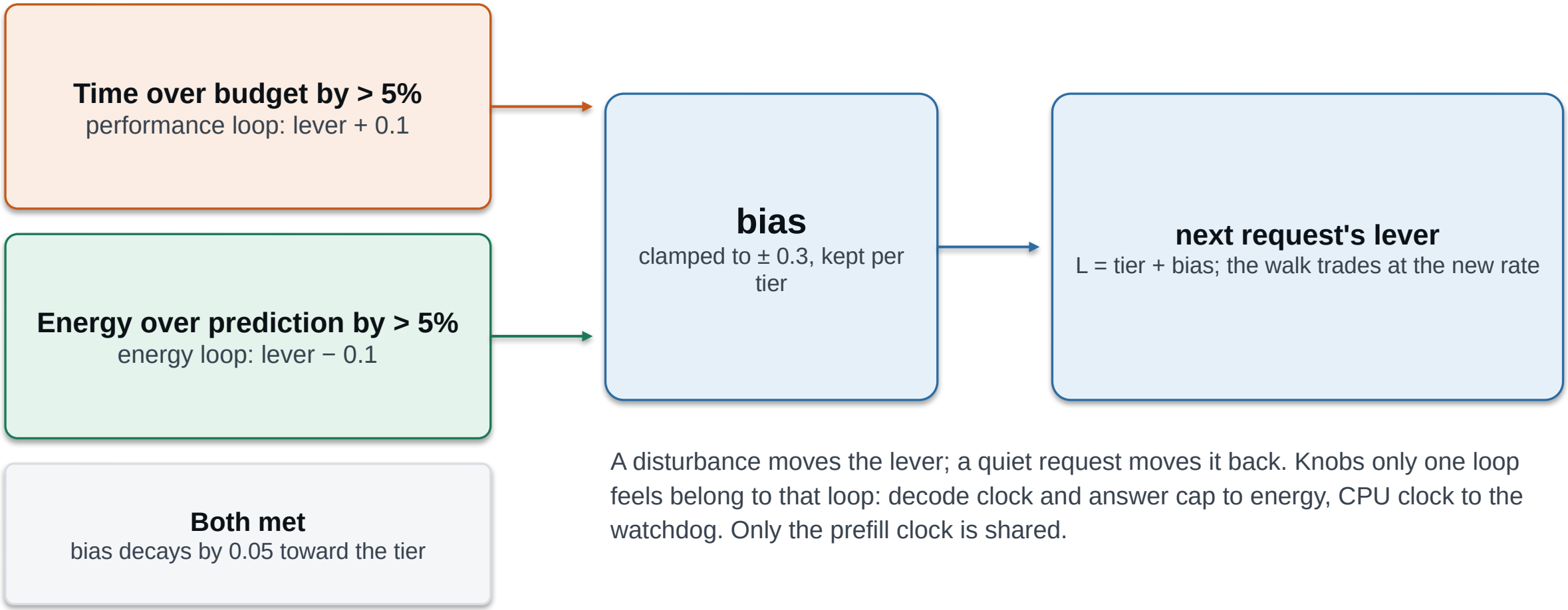


The plans are the split clock (prefill 1200, decode 902), then 902, then 726, with K fixed at 1024 and the answer cap from the tier.

The split plan is the two-configuration schedule that is optimal under a deadline (Kim, Imes, Hoffmann 2015). It saves 6% for 1.5% of time.

TWO LOOPS

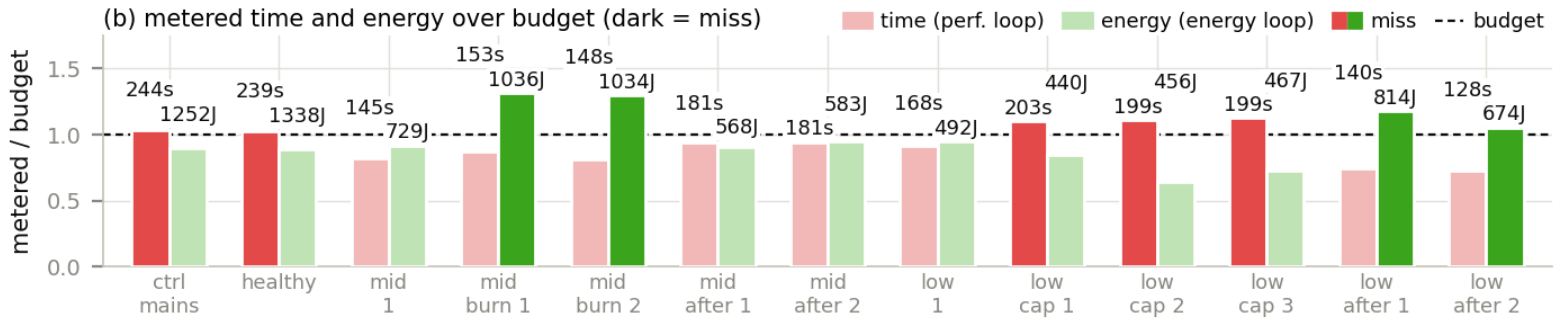
They act on the model's error, not on the raw meter



A disturbance moves the lever; a quiet request moves it back. Knobs only one loop feels belong to that loop: decode clock and answer cap to energy, CPU clock to the watchdog. Only the prefill clock is shared.

The JouleGuard coupling (Hoffmann, SOSP 2015). The cost table learns separately: EMA α 0.3, clipped to 10% per request.

Thirteen cooled requests with real disturbances

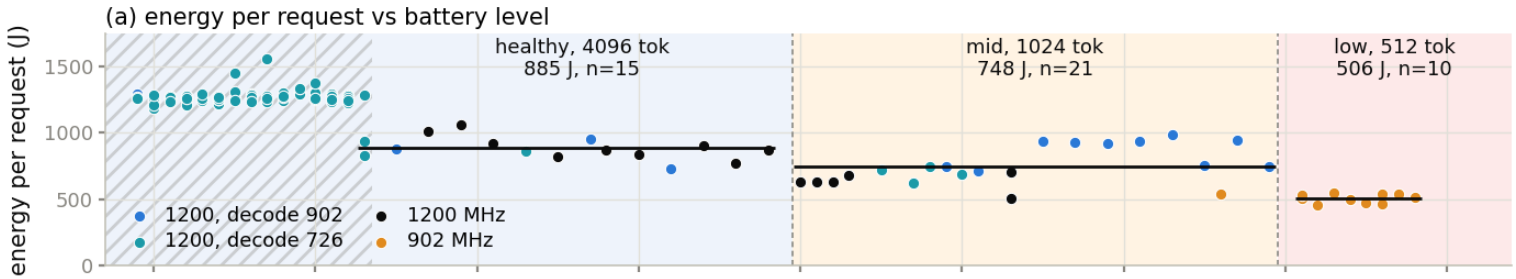


- Four idle cores spinning: energy 30% over budget. The energy loop moved the lever down twice; two clean requests decayed it back.
- An external 726 MHz cap 12 s into the request, what the vendor limiter does: time 10 to 12% over. The performance loop moved the lever up to its clamp, and back once the cap was gone.
- Dark bars are misses. Every miss was answered by the loop that owns it.

/tmp/loop_proof, 2026-09-05. Time budget and predicted energy, both + 5%.

THE REAL DISCHARGE

123 requests from 91% to 11%, nothing forced



885 J

healthy, cap 4096

748 J

mid, cap 1024

506 J

low, cap 512

−15%

mid vs healthy

−43%

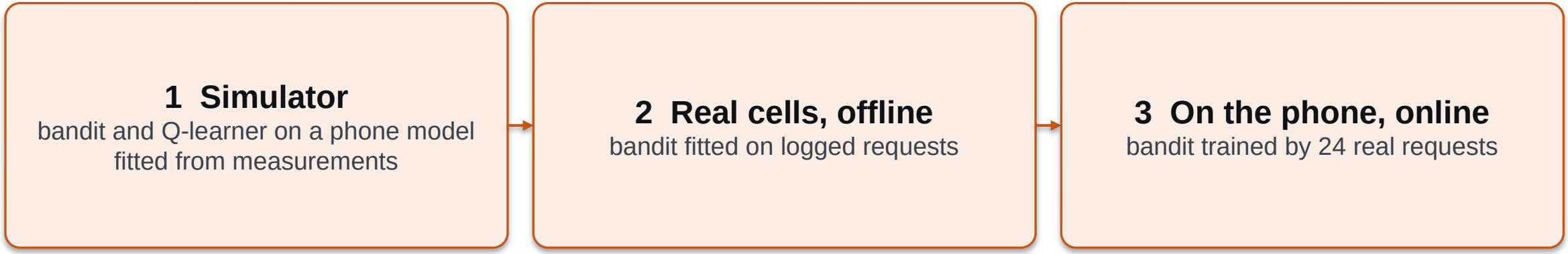
low vs healthy

Switched at 50% and 19%, on the first request past each threshold. Predictions within 2% after three requests on the battery.

/tmp/discharge_final, 2026-09-05/06. On the battery the OEM caps prefill at 726 MHz within a minute, so the caps carry the saving.

THE RL QUESTION

Would a learned policy beat the rule? Three experiments



Each step used less modeling and more measurement.

The answer never changed: one fixed plan per battery tier, and it is the plan the rule already picks.

EXPERIMENT 1 · SIMULATOR

Two agents, and how big each one is

Contextual bandit
state: battery bucket (4) × charging (2) = 8 contexts
arms: 9 (cache 342 / 688 / 1379 cells × clock 883 / 1267 / 1632 MHz)
table: 72 values · ε = 0.15, sample-mean update

Tabular Q-learning
state: battery (4) × charging (2) × DDR temperature (4) × throttled (2) = 64
arms: the same 9
table: 576 values · ε 0.15, α 0.15, γ 0.95

Reward: battery-weighted quality, throughput and energy, minus 0.25 when throttled. Training: 60 episodes × 6 seeds, each a discharge from 15%.

Result: the bandit converged to the best fixed arm at every battery level. The Q-learner never beat it. The simulator's thermal cliff never binds, so this is the weakest of the three.

energy_rl/agents.py, sim.py. Simulator fitted from the GPU cache axis, the CPU clock axis and the CPU soak (τ 221 s).

EXPERIMENT 2 · REAL CELLS, OFFLINE

Every logged request is one sample: tier, plan, metered reward

reward = w_q · quality − w_t · time / T_ref − w_e · energy / E_ref, with the tier's weights: healthy (0.55, 0.40, 0.05), mid (0.35, 0.20, 0.45), low (0.20, 0.10, 0.70)

GPU arm	healthy	mid	low
1200 MHz	best, P 1.00		
902 MHz		best, P 1.00	P 0.05
726 MHz			best, P 0.95
the rule picks	1200	902	902

- Agrees with the rule at healthy and mid.
- At low it takes 726 MHz: 3% less energy for 21% more time. The rule's stopping rule refuses that step.
- That is the weights against the marginal criterion, a product choice, not data against the rule.

energy_rl/bandit_real.py; P(best) from 2000 bootstrap resamples of the real cells. CPU arms: K = 1024 best at every tier.

EXPERIMENT 3 · ON THE PHONE

One real request per pull

3
arms: GPU 1200, 902, 726 MHz

3
contexts: battery tiers, cycled

0.25
ε, greedy otherwise

9 + 15
warm-start pulls + free pulls

Reward from the meter: $w_q - w_t \cdot T/T_{ref} - w_e \cdot E/E_{ref}$. Each pull: cool the phone, pin the CPU, run 9737 prompt + 1024 output tokens, meter it.

24
requests

15.7 kJ
spent learning

70 min
of inference

149 min
of phone time with cooling

run_bandit_online.py, /tmp/bandit_online, 2026-09-03. The seed drew no exploration step after the warm start.

EXPERIMENT 3 · RESULT

What it learned: one value per battery tier and arm

mean reward per pull (higher is better); pulls in brackets; the greedy arm is shaded

	1200 MHz	902 MHz	726 MHz	greedy arm	the rule
healthy	**+0.108 (6)**	+0.004 (1)	−0.099 (1)	1200	1200
mid	−0.305 (1)	**−0.237 (6)**	−0.290 (1)	902	902
low	−0.605 (1)	**−0.456 (4)**	−0.458 (3)	902 (726 within 0.002)	902

Read across a row: the tier's best arm. Every row ends where the rule already sits. The 726 arm at low is a tie with 902: 20 J less for 37 s more, which the rule's exchange rate refuses.

13 / 15

free pulls chose the rule's arm

15.7 kJ

spent to learn it

arm	J / request	s / request
1200 MHz	802	139
902 MHz	586	181
726 MHz	566	218

Same plans as the rule, so the same energy per request.

/tmp/bandit_online/log.txt: values after pull 23; pooled cost per arm over the 24 pulls, 9737-token prompt, 1024 output tokens.

RL VERSUS THE RULE

Same plans. Different costs, different failure modes

	rule: table + lever + loops	learned bandit
reaching the plan	one measured campaign	24 pulls, 15.7 kJ, 149 min
a new prompt shape	predicted from per-token costs	new warm start
cable to battery	table re-learned in 3 requests (36% → 2%)	new pulls per context
a disturbance	loops correct the lever, then decay	absorbed as the arm's cost
temperature	watchdog ladders at 2 Hz	not in the state; adding it did not help

The measurements removed the case for learning a policy. The case for learning the table stayed.

Learn the costs, not the policy

- Energy and time per plan do not depend on the state of charge. Only the weighting does, and the weighting is a design choice.
- The whole landscape is three thresholds and one stopping rule. Every agent rediscovered it.
- What drifts is the table, so the table learns: clipped EMA from every request, no bad plans explored on the user's battery.
- Learning a policy would earn its place only for something the meter cannot see, such as a user's tolerance for waiting.

SUMMARY

Energy-aware µKV in four lines

- Two knobs carry the saving: the GPU clock and the answer length. Cache and CPU clock do not.
- One lever from the battery tier, two loops on model error, one learned cost table.
- On a real discharge the lower tiers cost 15% and 43% less per request, and the GPU is used whenever it exists.
- Three RL experiments, from a simulator to 24 real requests on the phone, all converged to the rule's plans at a cost the rule does not pay.